

On Redundancy in Alloy Models

Soaibuzzaman
Bauhaus University Weimar
Weimar, Germany

Jan Oliver Ringert
Bauhaus University Weimar
Weimar, Germany

Abstract

Alloy is a modeling language for structural and behavioral properties combining first-order relational and temporal logic. Alloy models include structural and temporal constraints. Redundancy is a property in a constraint system where removing a redundant constraint does not impact the assignments satisfying the system. Intuitively, redundancy may degrade maintainability and potentially increase the cost of analyses.

We investigate and formally define redundancy for Alloy models. We implement a check to determine a constraint's redundancy, present an algorithm to find a maximal redundant subset of constraints, and, given a redundant constraint, we compute an explanation for its redundancy as a minimal subset of constraints that make it redundant. We evaluate our algorithms on a large corpus of Alloy models from various sources. Our evaluation assesses the prevalence of redundancy in Alloy models, the sizes of explanations, and the computational cost of our algorithms.

ACM Reference Format:

Soaibuzzaman and Jan Oliver Ringert. 2026. On Redundancy in Alloy Models. In *ACM/IEEE 29th International Conference on Model Driven Engineering Languages and Systems (MODELS 2026)*, October 04–09, 2026, Málaga, Spain. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3822455.3830324>

1 Introduction

Alloy [16–18] is a textual modeling language based on first-order relational logic and temporal logic. Alloy models declaratively express structures and behaviors of software designs. The Alloy Analyzer [2] provides various analyses for analyzing, finding, and visualizing instances or counterexamples for Alloy models. Analyses are possible in a bounded scope and fully automated by translations to SAT solvers and temporal model-checkers, making Alloy a popular lightweight formal method [18].

Alloy has been applied to modeling software designs [29, 51], code testing, debugging, and repair [12, 33, 50], and to analyze security properties [4, 45].

The Alloy language provides facilities for modeling structures based on signatures and their fields, where Alloy instances will populate signatures with atoms of unique identities and fields with relations over these atoms. Constraints on atoms and relations that constitute valid instances are expressed in predicates, functions, and facts. Behavior is modeled by first marking some signatures and fields as mutable and second by expressing temporal relations

between different states of the system, again in predicates, functions, and facts. Alloy models, even in their bounded scopes, typically allow many instances, and one proclaimed benefit of Alloy is the ability to browse them in the Alloy Analyzer [18].

In this work, we investigate the redundancy of constraints expressed within Alloy models. Intuitively, a redundant constraint is one that can be removed from an Alloy model without changing the set of instances that the Alloy model exhibits. On the one hand, redundancy may be seen as quality issues of models with a potential negative impact on maintainability and analysis cost. In the sense of code smells [23, 34] similar issues have been investigated, e.g., as vacuity for model-checking and synthesis [6, 11, 14, 27] or redundant multiplicity bounds in class diagrams [5]. On the other hand, redundancy might be intentionally introduced, e.g., to validate one's intention.

We formally define redundancy of constraints in Alloy models. We present algorithms to detect and explain it. Using our implementation, we investigate the prevalence of redundancy, the amount of redundant elements, the size of computed explanations, and the impact of removing redundant constraints on analysis times on a comprehensive dataset of publicly available Alloy models. We have further conducted an exploratory user study with 6 users in 1-to-1 sessions of ~45min each with our tool to investigate their perception of redundancy and to evaluate the tool support we devised.

Section 2 presents an example of redundant constraints. We present background in Sect. 3 and formal definitions of redundant constraints in Sect. 4. Section 5 gives an overview of our algorithms and Sect. 6 contains our evaluation. We conclude in Sect. 7.

2 Example

A team of engineers has created a basic model of teaching modules shown in Lst. 1, top. Signature `Person` has a field `teaches` (Lst. 1, l. 2) to express that any person may teach any number (multiplicity `set`) of modules. Signature `Module` has a field `teacher` referencing the one (default multiplicity, l. 6) `Person` teaching the module. The team added some restrictions based on their domain knowledge written in facts in Lst. 1, ll. 9–13. First, they state that the teachers of modules (relational join of `Module` with binary relation `teacher : Module × Person`) are professors (l. 10). Second, they state that the relations defined by fields `teacher` and `teaches` are inverse (i.e., they form bidirectional links, l. 12) and that all existing modules must be within the modules taught (l. 13).

The team analyzes the model for redundant constraints and discovers that Lst. 1, l. 13 is redundant. As this redundancy is not obvious to the team, they ask the tool to produce an explanation consisting of the constraints in l. 10 and l. 12: the teachers of modules are professors (l. 10) and the `teacher` and `teaches` relations being inverse (l. 12) ensures that all modules are taught (the redundant constraint in l. 13).



This work is licensed under a Creative Commons Attribution 4.0 International License. *MODELS 2026, Málaga, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2809-9/2026/10

<https://doi.org/10.1145/3822455.3830324>

```

1 abstract sig Person {
2   teaches : set Module
3 }
4 sig Professor extends Person {}
5 sig Module {
6   teacher : Person // default multiplicity one
7 }
8
9 fact teachersAndTeaching {
10   Module.teacher = Professor} // profs teach modules
11 fact teachersOfModules {
12   teacher = ~teaches // teachers teach their modules
13   Professor.teaches = Module} // no dangling modules
14 run {some Person} for 2 // browse example scenarios
15
16 + // extended later with additional signatures
17 + sig Student, Lecturer extends Person {}
18 + fact studentsDontTeach {
19   no s : Student | some m : Module | m in s.teaches}
20 + check {no Student.teaches} // confirm students don't teach
21 + run {some Lecturer.teaches} // show teaching lecturers

```

Listing 1: Two versions of an Alloy model of teaching modules (version 1: ll. 1-14, version 2: ll. 1-21)

In later stages of modeling, the team adds Lst. 1, ll. 16-21. They have added further signatures `Student` and `Lecturer` extending signature `Person`. They also added a fact that there is no student that teaches any module (l. 19). It turns out that this new constraint is redundant. The redundancy is again explained by the constraints in l. 10 and l. 12 (same as explaining why all modules are taught – by professors).

Finally, a member of the team wants to see instances where lecturers are teaching and adds the `run` command in Lst. 1, ll. 21. No instances are found.

What does this example teach us? Maybe, it is convincing enough that redundant constraints are easily formulated and added to models. Maybe the example shows that these are not easily spotted and require tool support to detect and explain. But, on the other hand, the two redundant constraints did not do any harm – up until l. 21. Lecturers cannot teach modules (l. 21) as all are taught by professors (l. 13). However, simply removing the redundant constraint in l. 13 would (by definition of redundancy) not change anything. We also have to modify or delete the constraint in l. 10, which was part of both redundancy explanations. Thus, tools for detecting and explaining redundancy do not automatically produce fixes for modeling issues, but they might help in better understanding constraints and thus support debugging and improvement of models.

3 Preliminaries and Related Work

3.1 Alloy Language

We give a brief introduction of the Alloy language based on the available references [2, 17, 25].

Definition 3.1 (Alloy model abstract syntax, simplified from [2, 17, 32]). The abstract syntax of an Alloy model $m \in \text{Model}$ consists of

- $m.\text{sig}s$ a set of signatures, where each signature s has
 - $s.\text{name}$ a unique, qualified signature name
 - $s.\text{isVar}$ a flag for being mutable
 - $s.\text{mult}$ a multiplicity **lone**, **some**, or **one**

- $s.\text{parent}/s$ a declaration of parent signatures¹
- $s.\text{fields}$ a set of fields, where each field f has
 - * $f.\text{name}$ a name unique for the signature
 - * $f.\text{isVar}$ a flag for being mutable
 - * $f.\text{mult}$ a multiplicity **lone**, **some**, **one**, or **set**
 - * $f.\text{expr} : \text{Expr}$ an expression defining the field's values (deriving a type)
- $s.\text{facts} : \text{list of Expr}$ a list of signature facts
- $m.\text{facts} : \text{list of Expr}$ a list of all facts of the model
- $m.\text{cmds}$ a set of commands, where each command cmd has
 - $\text{cmd}.\text{pred} : \text{Expr}$ the analyzed predicate or assertion
 - $\text{cmd}.\text{isCheck}$ an indicator for check or run command
 - $\text{cmd}.\text{scopes}$ scopes for each signature

Alloy provides a rich expression language indicated in Def. 3.1 by abstract syntax type *Expr* with Boolean, relational, arithmetic, and temporal operators (see [17, App. B], [3, 25]). Notice that the abstract syntax of Def. 3.1 does not mention predicates, functions, or assertions. These language elements are referenced either in signature facts, model facts, or the predicate of commands. As a simplification, we use $\text{cmd}.\text{pred}$ to denote the predicate being analyzed (also called assertion iff $\text{cmd}.\text{isCheck} = \text{true}$).

For an Alloy model $m \in \text{Model}$ and a command $\text{cmd} \in m.\text{cmds}$, we denote by $\llbracket m \rrbracket_{\text{cmd}}$ the elements of the semantics of model m when executing command cmd . These elements are called Alloy instances as sketched in Def. 3.2 and Def. 3.3.

Definition 3.2 (Alloy instance, simplified from [17, 25]). Given $m \in \text{Model}$ and a command $\text{cmd} \in m.\text{cmds}$, an instance $i \in \text{Inst}$ of m for cmd is an infinite sequence of states i_j for $j \in \mathbb{N}$ (represented by a finite prefix of length $k \geq 0$ and a finite loop of length $l > 0$) where for each state i_j , signature $s \in m.\text{sig}s$, and field $f \in s.\text{fields}$

- $i_j.\text{atoms}(s)$ is a set of atoms of each signature s bounded by $\text{cmd}.\text{scopes}$,
- $i_j.\text{rel}(f)$ is a relation of atoms satisfying $f.\text{mult}$ and $f.\text{expr}$
- sets of atoms and relations respect inheritance and subset relationships

and for consecutive states i_j and i_{j+1} immutable elements remain

- $\forall s \in m.\text{sig}s : \neg s.\text{isVar} \Rightarrow i_j.\text{atoms}(s) = i_{j+1}.\text{atoms}(s)$,
- $\forall s \in m.\text{sig}s, f \in s.\text{fields} : \neg f.\text{isVar} \Rightarrow i_j.\text{rel}(f) = i_{j+1}.\text{rel}(f)$,

and for the whole instance i

- all facts in $m.\text{facts}$ are satisfied,
- for all signatures $s \in m.\text{sig}s$, all signature facts $s.\text{facts}$ are satisfied for each atom of s , and
- if $\text{cmd}.\text{isCheck}$ then the assertion $\text{cmd}.\text{pred}$ is violated, else the predicate $\text{cmd}.\text{pred}$ is satisfied

Details about the evaluation of Alloy's temporal relational logic, including its operators and formal semantics, are provided in [25]. Note that facts and signature facts without temporal operators are implicitly wrapped into a temporal **always** operator and thus must be evaluated on infinite sequences of states. Without temporal operators, instances consist of an empty prefix and a single loop

¹Alloy's type signatures have a single parent $s.\text{parent}$ and Alloy's subset signatures have a set of parents $s.\text{parents}$

state, i.e., the temporal semantics of Alloy 6 subsumes the non-temporal semantics of previous Alloy versions. Instances of check commands that violate assertions are also called counterexamples.

Definition 3.3 (Alloy model semantics, simplified from [17]). Given a well-formed Alloy model $m \in Model$ and a command $cmd \in m.cmds$, the semantics of m and command cmd denoted by $\llbracket m \rrbracket_{cmd} \subseteq Inst$ are all instances of the model for command cmd .

3.2 Related Work

Quality Analysis of Alloy Models. Several studies examine the quality of Alloy models from different perspectives. On the fault localization and repair side, Wang et al. introduced AlloyFL [22, 48] and ARepair [47], combining spectrum- and mutation [19, 39]-based localization with program synthesis [30]. Subsequent work has expanded the repair landscape: FLACK [53] uses counterexample-guided localization, BeAFix [7] and ICEBAR [8] apply bounded-exhaustive and iterative search strategies, and Cerqueira et al. [10] extended repair to temporal Alloy 6 models. While fault localization addresses models with violated assertions, our work on redundancy addresses syntactically and semantically correct models.

On the semantic analysis side, Ringert and Wali provide semantic differencing, refinement, extension, and equivalence checks [32] for Alloy, and Wu and Sullivan propose AlloyASG [49] for compact predicate comparison. Our redundancy check is a semantic equivalence comparison between a model and the same model with a subset of constraints removed.

Empirical studies by Jovanovic and Sullivan [20] and Soaibuzzaman et al. [36] have analyzed edit histories to characterize the common mistakes users make when evolving Alloy models. Our user study (Sect. 6.5) and RQ1 evaluation complement this by showing that redundancy is highly prevalent in real-world models. Furthermore, we observe that redundancy introduced early in a model’s history tends to carry over across revisions, causing maintenance issues as models evolve.

Vacuity and Redundancy in Formal Specification. A related concept to redundancy is vacuity [6, 11]. In terms of Alloy models, an expression in a model is vacuously satisfied by an instance iff changes to the expression do not affect the satisfaction of the model by that instance. Fisman et al. [14] have lifted vacuous satisfaction of LTL formulas by models² to various kinds of inherent vacuity of LTL formulas. Their inherent vacuity framework includes redundancy [14, Example 7], e.g., Chockler and Strichman [11] remove redundant LTL constraints from formulas for better analyses, and Maoz and Shalom [27] detect it efficiently in GR(1) specifications for reactive synthesis. This special case maps our redundancy checks in the inherent vacuity framework of Fisman et al. [14]. Investigating other variants of inherent vacuity, they propose, might be worthwhile future work.

From a broader software quality perspective, redundancy in Alloy models constitutes a specification-level software smell [34] that compromises maintainability and performance (Sect. 6, RQ5) without causing immediate failures. This aligns with Thoben’s and Wehrheim’s [42] motivation, who similarly identify group redundancy (analogous to our subset redundancy) as a quality

defect in code contracts. Interestingly, Taibi et al. [40] concluded in an empirical study that developers, although agreeing that smells are generally harmful, consider them less harmful when provided with concrete examples. Our user study (Sect. 6.5) reveals that participants widely acknowledged that accidental redundancy is harmful, yet noted that deliberately introduced, tool-confirmed redundancy can serve as an effective mechanism for validating their mental models.

Explanation Approaches. Explaining why a constraint is redundant is closely related to the problem of finding a minimal unsatisfiable core. Our native explanation method builds directly on the core extraction technique introduced by Torlak et al. [43], leveraging the raw UNSAT core generated by the underlying SAT solver. To mitigate the verbosity of these raw cores, our general approach (Sect. 5.3) utilizes the *ddmin* [52] minimization technique. While our current implementation is based on *ddmin*, it is inherently algorithm-agnostic and can be integrated with alternative minimization approaches, such as QUICKXPLAIN [21]. Beyond these approaches, Dyer et al. [13] demonstrated the cognitive value of instance-based explanation. Building on their insights, a promising direction of future work is to complement our explanation with concrete witnesses (e.g., generating instances that satisfy the constrained model while explicitly violating the isolated redundant constraint).

4 Redundancy in Alloy Models

We assume two operations on the abstract syntax of Alloy models $m \in Model$. First, operation $cons : Model \rightarrow 2^{Expr}$ extracts all constraints from a model m , including top-level facts $m.facts$ and signature facts $s.fact$ for $s \in m.sigs$. Second, operation $rem : Model \times 2^{Expr} \rightarrow Model$ produces a new model by removing constraints from an existing model.

Our guiding intuition of redundancy is that removing a redundant constraint has no impact on the instances of a model for a command (see Def. 3.2 of instances). This intuition leads us to the definition of local redundancy in Def. 4.1.

Definition 4.1 (Local redundancy). Given an Alloy model m and a command $cmd \in m.cmds$, a constraint $c \in cons(m)$ is locally redundant, iff $\llbracket m \rrbracket_{cmd} = \llbracket rem(m, c) \rrbracket_{cmd}$.

Note that instances of Alloy models m and the semantics in Def. 3.3 always refer to a command $cmd \in m.cmds$. The term local in Def. 4.1 reflects this restriction to focus on a single command. We extend local redundancy to global redundancy in Def. 4.2 by requiring a constraint to be redundant for all commands in $m.cmds$. As a trivial example of the difference, consider the example from Lst. 1, ll. 1–14 with an added command `run {no Person}`. For the original command in Lst. 1, l. 14 only the constraint in l. 13 is redundant, however, for the new command `run {no Person}` all constraints (l. 10, l. 12, and l. 13) are redundant as the new command ensures that the relations `teaches` and `teacher` must be empty in all instances.

Definition 4.2 (Global redundancy). Given an Alloy model m , a constraint $c \in cons(m)$ is globally redundant, iff $\forall cmd \in m.cmds : \llbracket m \rrbracket_{cmd} = \llbracket rem(m, c) \rrbracket_{cmd}$.

²Here: Models in the sense of model-checking, i.e., a reference system to analyze.

Intuitively, removing globally redundant constraints does not change the semantics of a model for any of its commands. In the example of Lst. 1, ll. 1-14 with an added command `run {no Person}`, only the constraint in l. 13 is globally redundant, and the two constraints in l. 10 and l. 12 are locally redundant for the new command.

Based on Def. 4.2 of global redundancy, all globally redundant constraints are also locally redundant, but not necessarily the other way around, as demonstrated in the preceding example.

It is easy to see that some constraints are trivially redundant, e.g., the constraint `Person = Person` is always true and thus has no effect on instances.

Interestingly, some constraints are redundant for one scope but not for another. Consider a new fact `#Person < 3` stating that there are less than 3 atoms of signature `Person` in any instance of Lst. 1 and the `run` command in l. 14 with scope 2. This constraint is redundant as scope 2 allows 0, 1, or 2 atoms of `Person` per instance. If the scope in l. 14 was increased to 3, the fact `#Person < 3` would no longer be redundant.

Note that individual redundancy of constraints does not imply their joint redundancy, i.e., if constraints c and c' each are redundant, it might be that constraint $c \text{ and } c'$ (the set $\{c, c'\}$) is not redundant. As an example, consider a variant of Lst. 1 where we change the keyword `check` in l. 20 to `fact` and delete the lines that made l. 19 redundant (l. 10 and l. 12). Now the constraints about students not teaching modules in l. 19 and l. 20 are semantically equivalent and l. 19 makes l. 20 redundant and vice versa. However, removing both constraints changes the semantics of the model.

This motivates Def. 4.3 of maximal globally redundant subsets of constraints.

Definition 4.3 (Maximal globally redundant subset). Given an Alloy model m , a set of constraints $C \subseteq \text{cons}(m)$ is a maximal globally redundant subset, iff

- C is a globally redundant subset, i.e., $\forall \text{cmd} \in m.\text{cmds} : \llbracket m \rrbracket_{\text{cmd}} = \llbracket \text{rem}(m, C) \rrbracket_{\text{cmd}}$, and
- C is maximal for this property, i.e., $\forall C'$ where $C \subset C' \subseteq \text{cons}(m)$, $\exists \text{cmd} \in m.\text{cmds} : \llbracket m \rrbracket_{\text{cmd}} \neq \llbracket \text{rem}(m, C') \rrbracket_{\text{cmd}}$.

Note the similarity of the definition of redundant subsets to redundant constraints from Def. 4.1. It is also straightforward to change this definition from maximal globally redundant subsets to maximal locally redundant subsets by fixing a command $\text{cmd} \in m.\text{cmds}$ up front.

Interestingly, if a `run` command or a `check` command is unsatisfiable, the complement of a maximal redundant subset is an UNSAT core (none of the redundant constraints are needed to ensure that $\llbracket \text{rem}(m, C) \rrbracket_{\text{cmd}} = \emptyset$). Practically, this leads to many locally redundant constraints for unsatisfiable models and commands.

4.1 Beyond Facts and Commands: redundancy of structural constraints?

Alloy is a rich modeling language that enables engineers to express constraints through language elements other than facts.

Multiplicities of signatures and fields introduce constraints on instances, e.g., field multiplicity `some` in Lst. 2, l. 6 requires the teaches relation to contain at least one tuple for each atom of `Professor`. Semantic extensions of redundancy to multiplicities

```

1 abstract sig Person {}
2 sig Student extends Person {
3   attends : set Module
4 }
5 sig Professor extends Person {
6   teaches : some Module
7 }
8 sig Module {}
9 sig University {
10  members : set Person
11 } {
12  members in Student + Professor // restrict members
13 }
14
15 fact {
16   all p : Professor | #p.teaches >= 1 // professors must teach
17 }

```

Listing 2: Redundant (signature) facts due to constraints from structural Alloy language elements

```

1 var lone sig Request {}
2 var lone sig Grant {}
3 fact requestsAreEventuallyGranted {
4   always (some Request implies eventually some Grant)
5 }
6 run {always no Request}

```

Listing 3: Example of redundant temporal constraints (in LTL terminology, the run command in l. 6 enforces a vacuous [6, 11] satisfaction of the fact in l. 4)

are not syntactically straightforward: intuitively, removing a redundant syntactic element from the Alloy model should remove redundancy; however, removing the multiplicity `some` would revert to the more restricted default multiplicity `one` of fields rather than the semantically intended, most permissive multiplicity `set`.

Similarly, inheritance between signatures introduces constraints, e.g., the declarations of the abstract parent signature `Person` in Lst. 2, l. 2 for signature `Student` and in l. 5 for signature `Professor` restrict atoms of `Person` to `Student` and `Professor` making this inheritance relation redundant to the signature fact in Lst. 2, l. 12. Again, this redundancy cannot easily be resolved syntactically due to the use of `Person` for `Student` and `Professor` in Lst. 2, l. 10 and Alloy's ignoring of `abstract` declarations of signatures when no child signatures are declared.

Note that in all cases where facts and commands are redundant to structural constraints, our definition and implementation take these constraints into account (see Sect. 5). However, structural constraints are never considered redundant.

4.2 Redundancy and Temporal Logic

The Alloy language supports temporal logic with mutable signatures and fields, e.g., the signatures `Request` and `Grant` are both mutable in Lst. 3. Alloy instances, as presented in Def. 3.2, are sequences of states where only mutable elements may change between consecutive states [9, 25], e.g., the atoms of signatures `Request` and `Grant` from Lst. 3 could be different between states.

Temporal relations between states are defined in facts and commands and are thus naturally included in our definition of redundancy from Def. 4.1 and Def. 4.2. In Lst. 3, the fact in line 4 is

redundant as it is vacuously satisfied since `Request` never has any atoms in any instance.

5 Detecting Redundancy

Given an Alloy model m and a command $cmd \in m.cmds$ the definitions of local and global redundancy require the decision(s) whether $\llbracket m \rrbracket_{cmd} = \llbracket rem(m, C) \rrbracket_{cmd}$, i.e., where a subset of constraints $C \subseteq cons(m)$ has been removed from m . Due to the structural similarity and the difference between m and $rem(m, C)$ only in constraints, this check is much simpler than a general semantic comparison [32] of arbitrary Alloy modules. We construct a new constraint shown in Eqn. (1) and check it for satisfiability in m where all other constraints are removed.

$$\bigwedge (cons(m) \setminus C) \wedge \neg \bigwedge C \quad (1)$$

Technically, we realize this check by creating a new Alloy model $m' = rem(m, cons(m))$ from model m with all constraints removed. We then create a new command cmd' from command cmd where we set $cmd'.pred$ to Eqn. (1), $cmd'.check$ to `false`, and $cmd'.scopes$ to $cmd.scopes$. We execute command cmd' using the Alloy Analyzer API. If the command execution returns `unsatisfiable`, then the constraints in C are redundant for m and cmd .

On Soundness and Completeness. Given model m , command $cmd \in m.cmds$, and constraints $C \subseteq cons(m)$, the check based on Eqn. (1) is sound in the sense that constraints C are redundant if the check passes. Intuitively, the unsatisfiability of the created command cmd' must originate from a conflict in Eqn. (1) as cmd' uses the same scopes as cmd . If the constraint in Eqn. (1) is satisfiable in m then at least one constraint $c' \in C$ is violated and thus model $rem(m, C)$ (left side of Eqn. (1)) permits a model that m (including c') does not permit, i.e., $\llbracket m \rrbracket_{cmd} \neq \llbracket rem(m, C) \rrbracket_{cmd}$. If Eqn. (1) is not satisfiable then $\llbracket m \rrbracket_{cmd} = \llbracket rem(m, C) \rrbracket_{cmd}$. However, the check is not complete, i.e., there are cases where the execution terminates with a higher-order quantification error. Consider adding the fact `fact {some x: set Person | Module.teacher = x}`. This fact contains higher-order quantification (variable x as a set of `Person` atoms). In the positive form, this higher-order quantification is supported by the Alloy Analyzer through implementation tricks, but the negation of the fact, as required in Eqn. (1) is not.

5.1 Preprocessing

Our definitions of the various types of redundancy rely on the operation $cons : Model \rightarrow 2^{Expr}$ to extract all constraints from a model m . It turns out that this operation has no direct counterpart on the Alloy API level and is not directly obtained from the abstract syntax of Alloy models (see Def. 3.1). We use two preprocessing steps on the abstract syntax of Alloy models m .

First, we iterate over all signatures $s \in m.sigs$ and transform each signature fact $f \in s.facts$ to a module fact $f' \in m.facts$. This transformation enables us to treat constraints (from facts and signature facts) in a uniform way. Our transformation deletes each fact $f \in s.facts$ and either wraps it into a universal quantification over atoms of s or into a `let` expression in case the signature s has multiplicity `one`. It then adds the wrapped fact to $m.facts$. As an example, the signature fact `members in Student + Professor` of signature `University` from Lst. 2, l. 12 would be transformed into the

model-level fact `fact {all u : University | u.members in Student + Professor}`³.

Second, we recursively flatten all constraints (abstract syntax type *Expr*) into conjuncts. We remove the pseudo-operator `NOOP`, and extract conjuncts from expression lists and binary expressions with operator `AND`.

Intuitively, this flattening step leads to a finer granularity of successive analyses of constraints. It also enables the detection of redundant constraints that might otherwise go undetected. In our initial example shown in Lst. 1, ll. 1-14, the fact `teachersOfModules` in ll. 11-13 is not redundant, but the second (implicit) conjunct `Professor.teaches = Module` is redundant, and our algorithms detect it.

Lower-Level Constraints. The Alloy language is translated into the syntactically reduced and simpler, lower-level constraint languages `Kodkod` [44] or its temporal logic extension `Pardinus` [24] for analysis. These lower-level constraint systems might be suitable for similar analyses of redundancy; however, a lot of structural information is lost. As an example, `Kodkod` [44] has no concept of inheritance and Alloy's inheritance between signatures is thus encoded in generic `Kodkod` constraints or `Kodkod` bounds. In favor of ease of traceability, we have decided to implement our analyses on the Alloy level rather than its lower-level solvers.

5.2 Computing a Maximal Redundant Subset

To maximize redundant subsets of constraints, we use classic minimization algorithms for delta debugging [52] and minimize the complement of the redundant constraints. These algorithms, e.g., `ddmin` [52], iteratively check different subsets to preserve a property, e.g., a failing test result or redundancy of constraints. We initialize the minimization algorithm with $m.facts$ after preprocessing. Our check criterion for local redundancy is shown in Alg. 1.

Algorithm 1 Implementation of the check criterion for minimizing $nonRedSet \subseteq m.facts$ after preprocessing

Require: $nonRedSet \subseteq m.facts$

- 1: $C \leftarrow m.facts \setminus nonRedSet$
 - 2: $nonRed \leftarrow \bigwedge nonRedSet \triangleright \text{TRUE}$ if $nonRedSet = \emptyset$
 - 3: $redundant \leftarrow \bigwedge C \triangleright \text{TRUE}$ if $C = \emptyset$
 - 4: $m' \leftarrow m, cmd' \leftarrow cmd \triangleright \text{copy model and command}$
 - 5: $cmd'.pred \leftarrow cmd.pred \wedge nonRed \wedge \neg redundant$
 - 6: $m'.facts \leftarrow \emptyset \triangleright \text{remove facts; they are in command now}$
 - 7: **if** `execute( $m', cmd'$ ) = unsatisfiable` **then**
 - 8: **return** `FAIL` $\triangleright$ *still redundant/entailed – dig deeper*
 - 9: **else** $\triangleright$ *instance found*
 - 10: **return** `PASS` $\triangleright$ *not redundant*
-

Algorithm 1 computes the set C (l. 1) of potentially redundant constraints by removing the candidate non-redundant set from all constraints. It then constructs the redundancy check as described in Eqn. (1) and conjuncts the command's predicate (l. 5) to update command cmd' . If this command is unsatisfiable Alg. 1 returns `FAIL` to indicate that the property to minimize for is still preserved,

³The implicit this resolution that enables the join `u.members` is handled by Alloy during parsing.

i.e., *nonRedSet* may be reduced further to entail the remaining, redundant constraints.

We can easily extend this implementation for maximizing a locally redundant subset to an implementation for globally ones. Our check criterion in Alg. 1 is modified to iterate over all commands and for each confirm unsatisfiability. In case an instance is found, PASS is returned immediately (as redundancy was not confirmed for at least one command), whereas the result FAIL is only returned after all executions of commands were found to be unsatisfiable.

Note that *ddmin* [52] is not the only algorithm for computing minimal sets wrt. a monotone property. Another popular alternative to *ddmin* is QUICKPLAIN [21] – both have their strengths and weaknesses [41]. Our work is independent of the particular minimization algorithm.

On Soundness and Completeness. Algorithm 1 wraps our sound but incomplete Alloy-based analysis of redundancy into a search for different subsets. Given sound and complete minimization algorithms (such as the ones we employ), the computation remains sound and incomplete (as our basic redundancy check is incomplete in case of higher-order quantification).

5.3 Explaining Redundant Constraints

After some initial experiments with prototypical implementations of redundancy detection, we noticed that it is not always easy to understand the reasons for the redundancy of constraints. We have thus implemented an explanation computation. Given a model m , command $cmd \in m.cmds$, and constraint $c \in m.facts$ (after preprocessing), an explanation for the redundancy of c is a minimal set of constraints $E \subseteq m.facts \setminus \{c\}$ that make c redundant.

The application of delta debugging minimization algorithms for computing explanations appears obvious. We show a check criterion for our *ddmin* [52] implementation in Alg. 2. Again, the algorithm is based on our check of redundancy from Eqn. (1), here, l. 2 encodes in predicate $cmd'.pred$ that the explanation E and the original command predicate $cmd.pred$ make constraint c redundant.

Algorithm 2 Criterion to compute minimal explanation $E \subseteq m.facts \setminus c$, that makes $c \in m.facts$ redundant for command cmd

Require: $E \subseteq m.facts \setminus \{c\}$

```

1:  $m' \leftarrow m, cmd' \leftarrow cmd \triangleright$  copy model and command
2:  $cmd'.pred \leftarrow \bigwedge E \wedge cmd.pred \wedge \neg c$ 
3:  $m'.facts \leftarrow \emptyset \triangleright$  remove facts, they are in command now
4: if  $execute(m', cmd') = \text{unsatisfiable}$  then
5:   return FAIL  $\triangleright$  still redundant/entailed – dig deeper
6: else  $\triangleright$  instance found
7:   return PASS  $\triangleright$  not redundant
```

We observe that the explanation of the redundancy of a constraint performs a minimization on an unsatisfiable formula (format of Eqn. (1)). Thus, we formulate an alternative explanation computation that directly leverages the native UNSAT-core computation of Alloy [43] as shown in Alg. 3.

Notice that the two algorithms compute different explanation outputs. Algorithm 2 computes a subset $E \subseteq m.facts$ while Alg. 3

Algorithm 3 Native, UNSAT-core-based explanation computation for redundancy of $c \in m.facts$ for command cmd

Require: $c \in m.facts$

```

1:  $m' \leftarrow m, cmd' \leftarrow cmd \triangleright$  copy model and command
2:  $cmd'.pred \leftarrow \bigwedge (m.facts \setminus c) \wedge cmd.pred \wedge \neg c$ 
3:  $m'.facts \leftarrow \emptyset \triangleright$  remove facts, they are in command now
4: return  $\text{unsat-core}(m', cmd')$ 
```

computes an UNSAT core. An UNSAT core may also include structural constraints and elements of Alloy models. We evaluate their difference in Sect. 6, RQ3.

Note that one could also attempt to explain non-redundancy of constraints, e.g., by computing an instance that satisfies all other constraints and violates the constraint to explain. Initial work has been carried out on studying information content of explanations based on instances [13].

6 Evaluation

6.1 Implementation

We have implemented the redundancy detection and explanation algorithms presented in Sect. 5 as an extension to the Alloy Analyzer, building upon Alloy version 6.2⁴. The implementation required modifier changes in the Alloy abstract syntax API, e.g., removing the `final` modifiers to allow the temporary mutation of otherwise immutable signature facts.

To make our contribution easily accessible, we seamlessly integrated these capabilities into the Alloy Analyzer’s graphical user interface (GUI). As illustrated in Fig. 1, we introduced dedicated controls that allow users to compute a globally maximal redundant set for the active model and immediately visualize the affected elements via syntax highlighting directly within the editor. Furthermore, we implemented an interactive, on-demand explanation feature: users can select any specific constraint within the editor to compute and display the root cause of its redundancy, independent of its inclusion in the global maximally redundant set.

Artifacts for this paper are publicly available on Zenodo [37]

6.2 Research Questions

Our evaluation investigates the following research questions:

- **RQ1** What is the prevalence of redundancy in Alloy models?
- **RQ2** What are the sizes of maximally redundant subsets?
- **RQ3** What are the sizes of explanations of redundancy for the general and native algorithms?
- **RQ4** What is the cost of computing maximally redundant subsets and explanations?
- **RQ5** What is the performance impact of globally redundant constraints on analyses?

6.3 Dataset

In particular, to investigate the prevalence of redundant constraints in Alloy models, our evaluation requires a large, representative

⁴Version based on commit <https://github.com/AlloyTools/org.alloytools.alloy/commit/0fa93b2c8da3673caaf8e697e46ecea966eab3b5> from Jun 13, 2025

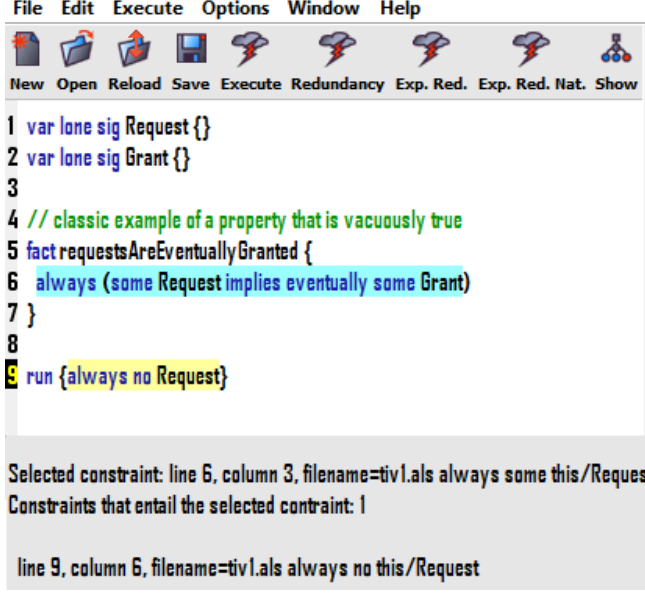


Figure 1: Integration of selected checks into the Alloy Analyzer user interface (top right buttons, see Sect. 6.1) currently showing the explanation (l. 9) for the redundant fact (l. 6)

dataset. As our subsequent analysis will demonstrate, certain existing datasets, e.g., the one of Alloy4Fun [26], have specific structural characteristics that can heavily skew redundancy metrics. To ensure a generalized evaluation, we compiled a novel dataset by mining public GitHub repositories for real-world Alloy models.

Alloy Model Discovery. We utilized the GitHub Search API to gather all publicly available and indexed repositories featuring the Alloy language extension (`.als`). Because the API imposes strict pagination and rate limits (a maximum of 100 results per call and 30 calls per minute⁵), we designed a search strategy utilizing diverse combinations of Alloy-specific keywords and language filters to maximize repository acquisition. Through this process, we successfully identified 1,124 repositories containing Alloy models on January 20, 2026.

Dataset Cleaning. From these repositories, we initially extracted 88,745 files with `.als` extension. We then applied a rigorous filtering pipeline to ensure quality and compatibility. First, we retained only syntactically valid models where at least the first command was successfully parsed by Alloy 6. To address syntactic incompatibilities between older Alloy models and version 6, we implemented an automated migration step that replaces single quotes (`'`) with underscores (`_`) when temporal operators are present.

Next, we deduplicated the dataset to remove redundant copies of well-known benchmarks (e.g., models from [1, 2, 28, 31, 38, 47–49] that frequently appeared across multiple repositories). Finally, we discarded models that failed during runtime due to unsupported features (such as higher-order quantification or recursive calls on `fun` or `pred`), bounded engine limitations, translation capacity errors,

Table 1: Characteristics of the Alloy models in the dataset

	Lines	Sigs	Facts	Flat Facts	Commands
25%	30	2	0	1	1
Median	68	5	1	5	1
75%	152	14	3	16	2
Max	1,248,673	508	262	709	178,103

or other unhandled exceptions. Following this cleaning process, we established a final dataset of 36,885 unique, analyzable Alloy models for our evaluation.

Dataset Characteristics. Table 1 summarizes the structural characteristics of the models in our refined dataset. We report on the quartiles for total lines, signatures, facts, flat facts (facts after flattening from Sect. 5.1), and commands. The median model length is 68 lines of code (LOC), with the 75th percentile reaching 152 LOC. In terms of structural complexity, the median model contains 5 signatures, 1 fact block, 5 flat facts, and 1 command. At the 75th percentile, these metrics scale to 14 signatures, 3 fact blocks, 16 flat facts, and 2 commands. The largest and most complex models in our dataset originate from the benchmarks presented in [15] and [35].

6.4 Experiments and Results

All computations were performed on an ordinary desktop computer with an Intel i7-7700K @ 4.2 GHz CPU and 32 GB RAM.

RQ1: Prevalence of Redundancy

To investigate the prevalence of redundancy (*RQ1*), we check for globally and locally redundant constraints as described in Sect. 5.

We initially evaluated a large dataset comprising 36,885 Alloy models. However, a significant portion of this dataset originates from Alloy4Fun [26]. The Alloy4Fun models consist of a small, high-quality, instructor-provided core and for each command a single student-written predicate. Given this particular nature, we did not detect any case of redundancy among the Alloy4Fun models and removed these from our main statistics. Removal yields a refined, representative corpus comprising 5,220 models. Within this refined dataset, we found that redundancy is a prevalent issue: 2,108 of the 5,220 models (40.9%) contain locally redundant constraints, and 1,388 (26.6%) exhibit globally redundant constraints.

These findings suggest that within Alloy models, redundancy occurs with moderate frequency, impacting more than one-third of the models locally and nearly one-fourth globally.

RQ2: Sizes of Maximally Redundant Subsets

To answer *RQ2*, we analyze the sizes of maximally redundant subsets across our dataset. First, we report the size of maximally locally (top) and globally (bottom) redundant sets expressed as the total number of redundant constraints, shown in Fig. 2. Both distributions show a clear long-tail pattern. In most affected models, redundancy is mostly localized, usually involving fewer than 10 redundant constraints. However, the extreme tail highlights a significant subset of models with extensive redundancy, comprising dozens or even hundreds of redundant constraints. A manual analysis showed that the models with the highest amount of redundancy were generated by automated model analysis tools, e.g., [46].

⁵<https://docs.github.com/en/rest/search/search?api=&apiVersion=2026-03-10>

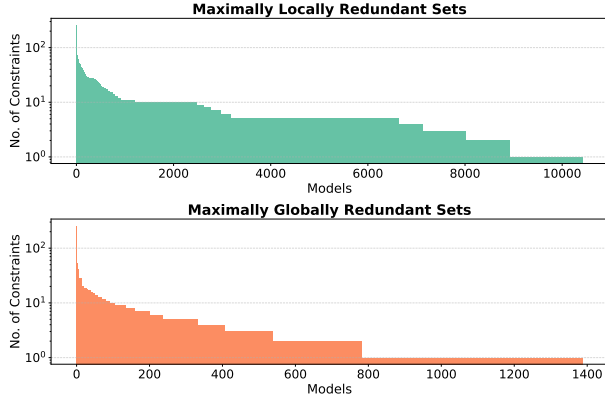


Figure 2: Distribution of sizes (measured by constraints) of maximally locally (top) and globally (bottom) redundant sets. The y-axis is plotted on a logarithmic scale to account for models with extreme redundancy.

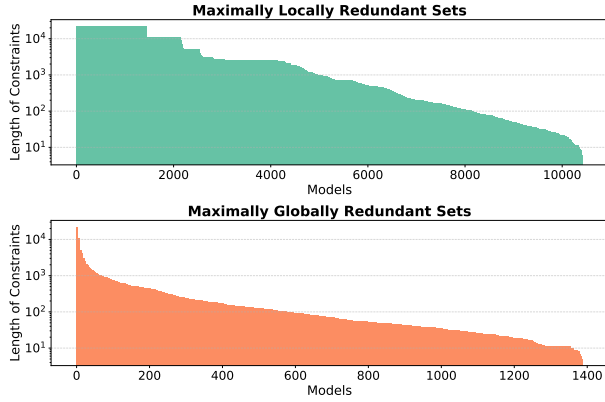


Figure 3: Distribution of total character length in redundant sets. The log y-axis shows that most redundancies are short, but some models have blocks over 10^4 characters.

As the length of a constraint might differ between Alloy models, we report the combined length in terms of characters of maximally locally (top) and globally (bottom) redundant sets in Fig. 3. The character-length distributions further emphasize the severity of redundancy in the outlier models. While most redundant sets are relatively short (i.e., between 10 and 1,000 characters), a substantial number of models contain redundant constraint blocks spanning several thousands of characters.

This indicates that redundancy in Alloy is not strictly limited to trivial, single-line duplicates; it often appears as large, complex blocks of logic that are semantically entailed by the rest of the model.

RQ3: Sizes of Explanations

To answer RQ3, we computed explanations for all constraints inside the previously computed maximally global redundant sets. We evaluate their sizes using both constraint count and explanation character length.

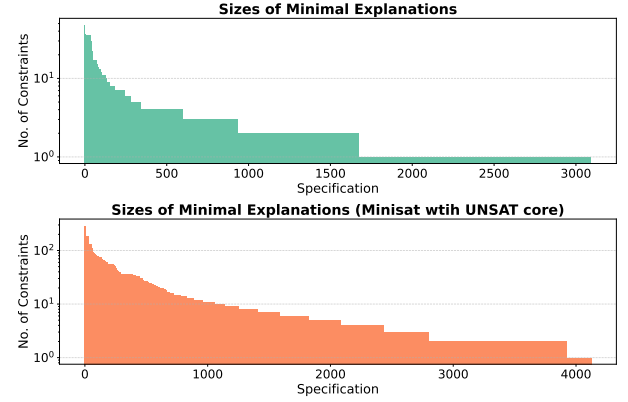


Figure 4: Histogram of sizes of minimal explanations of redundant constraints (y-axis indicates number of constraints)

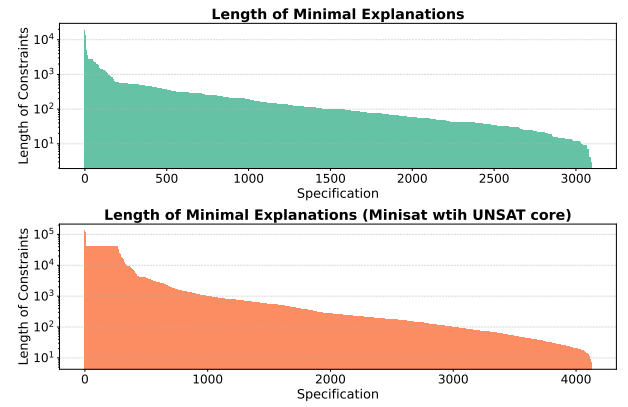


Figure 5: Histogram of length of minimal explanations of redundant constraints (y-axis indicates length in characters)

Fig. 4 presents histograms illustrating the sizes of minimal explanations. The distribution reveals a clear difference in verbosity between the two methods described in Sect. 5.3. Explanations generated by the general approach (top, Alg. 2) are highly concise, with the vast majority requiring fewer than 10 constraints and outliers capping out relatively early. Conversely, the native approach (bottom, Alg. 3) exhibits a long tail, with extreme outliers often exceeding 100 constraints. Since the length of a constraint used for explanation can vary between Alloy models, we report their combined length in terms of characters per explanation in Fig. 5. Measuring by character length reveals a similar trend: the general explanations are compact, whereas the native approach is significantly larger, resulting in extreme outliers that reach nearly 100,000 characters.

The general approach computed an empty explanation, i.e., a reason for redundancy outside of the set of constraints, in 1,064 of 4,157 cases. To compare the general explanation computation (Alg. 2) and the native explanation computation (Alg. 3) approaches, we calculated ratios between the number of elements in explanations and their lengths. In 57.31% of the cases, the native approach

Table 2: Ratios of the time finding local/global redundancy vs. checking each constraint iteratively and computing general vs. native explanations

	Max. Red. Local	Max. Red. Global	Expl. General/Native
25%	2.01	1.05	0.05
Median	2.30	1.48	0.06
75%	2.54	1.95	4.56

Table 3: Absolute times of computing local redundancy, global redundancy, general explanations, and native explanations (all times in seconds)

	Max. Red. Local	Max. Red. Global	General Expl.	Native Expl.
25%	0.21	0.35	1.87	0.64
Median	0.68	2.10	1.99	40.73
75%	1.22	16.0	2.40	41.31

computed explanations with more elements than the general approach. In 52.46% of the cases, the native approach computed longer explanations than the general approach. In 75% of the cases, the native approach computed eight times larger explanations and 10.27 times longer explanations (ratios computed by dividing *native size* / *general size* for all pairs where *general size* > 0).

When the general approach computed explanations with 0 elements, the native approach successfully provided an explanation in 1,058 cases (representing 25.66% of the overall dataset). For these specific cases, the first quartile and the median sizes of the native explanations are 2 and 4 elements, respectively, while their lengths are 141.25 and 341 characters, respectively.

These results highlight a clear trade-off between conciseness and computational robustness in explanation generation. The general approach minimizes the explanation size but often fails to identify sources of redundancy. The native method guarantees a solution but produces large, complex blocks. Interestingly, when the general approach fails (yielding 0 elements), the native explanation is surprisingly short (median of 4 constraints). Note that in these cases the explanations must be based on structural elements of the model making it impossible to violate the constraint to explain.

RQ4: Cost of Computations

To answer *RQ4*, we have computed ratios of the time it takes to find a maximally redundant subset divided by the time it takes to determine all redundant constraints by iterating over them one by one. We chose the iteration as a naive baseline. Note that the maximally redundant subsets are more informative than the set of all redundant constraints. We present these ratios in Table 2 and the corresponding absolute execution times in Table 3. The quartiles are computed for each Alloy model and command for local analyses (first column) and for each Alloy model for global analyses (second column). Out of 36,885 models in our dataset, the computation of maximally redundant subsets was successfully

Table 4: Ratios of Alloy analyses with redundant constraints to analyses without these

	Time Ratio	Primary SAT variables	SAT variables
25%	1.43	1.00	1.00
Median	1.79	1.00	1.06
75%	2.30	1.00	1.61

completed in the majority of cases. Timeouts were observed in only 2.9% (1,097 models) for local analyses and 3.1% (1,148 models) for global analyses.

As shown in the first column of Table 2, computing the more informative maximally redundant subsets is proportionally more expensive than computing all redundant constraints, with a median ratio of 2.30 and a 75th percentile of 2.54. This gradual increase indicates that the relative overhead remains roughly constant for the majority of models. However, as Table 3 shows, this overhead results in negligible absolute delays in practice; 75% of local analyses finish in under 1.22 seconds. Similar relative numbers can be observed when computing globally maximal subsets (median 1.48, 75% 1.95), while the absolute times increase but remain practical, with 75% of models computing global redundancy in 16 seconds or less.

Finally, we show quartiles of the ratios of our explanation generation methods in the third column of Table 2. Surprisingly, the general approach (Alg. 2) is often much faster, taking only a fraction (0.06×) of the native execution (Alg. 3) time at the median. The absolute times clearly illustrate that the general approach demonstrates high consistency, taking approximately 2 seconds across the 25th to 75th percentiles. In contrast, native extraction times vary significantly, increasing from 0.64 seconds at the 25th percentile to over 40 seconds at the median. However, this trend reverses in the upper quartiles. By the 75th percentile, the general approach becomes 4.56 times slower than the native approach. This large variation is expected. While our general approach can quickly resolve simple constraints, the *ddmin* algorithm requires repeated solver calls, causing longer computations on complex models. This confirms the trade-off identified in *RQ3* that the general approach yields more concise explanations but can cause increased execution times and occasional timeouts compared to a single-pass native extraction.

RQ5: Performance Impact of Redundant Constraints

Finally, to address *RQ5*, we evaluated the execution overhead on affected Alloy models programmatically. For each model, we selected a single command (defaulting to the first available command) and translated it using the Kodkod backend. We recorded the solving time and generated SAT variables for the full model, and then immediately repeated this translation and solving process with the identified redundant constraints removed. Table 4 shows the quartiles for the ratios between these two executions (Original / Without Redundancy). A ratio greater than 1.0 indicates that redundancy increases the metric.

The data shows a significant performance penalty associated with redundancy. Keeping redundant constraints makes the analysis

1.79 times slower at the median and 2.30 times slower at the 75th percentile.

To understand this slowdown, we analyzed the size of the generated Boolean satisfiability problems. While the number of primary SAT variables remains completely unaffected (maintaining a constant ratio of 1.00), the total number of SAT variables increases by a factor of 1.06 at the median and 1.61 at the 75th percentile. This indicates that translating redundant formulas introduces additional structural auxiliary variables into the Conjunctive Normal Form (CNF). This increased formula complexity forces the underlying SAT solver to do significantly more work, demonstrating that removing global redundancy is a critical step for optimizing Alloy’s analysis performance.

6.5 User Study

We conducted a small user study to qualitatively analyze the role of redundancy in Alloy models with six participants: two experienced Alloy users (*E1*, *E2*) and four novices (*N1*–*N4*). The experts had used Alloy in research and teaching for ≥ 10 years, while the novices had taken at least one graduate course involving formal methods and had submitted Alloy-related assignments. Each remote session lasted 40–50 minutes, was screen-shared, video-recorded, and transcribed for analysis.

Each session followed a structured protocol of three tasks and a closing discussion. Participants received access to the custom Alloy Analyzer build described in Sect. 6.1.

Task 1 (warm-up). Participants were given the Alloy model of Lst. 1 (ll. 1–14) and asked to extend it in two steps: first, add a Student signature that cannot teach modules; second, add a Lecturer signature that can teach modules. This task was designed to show participants the practical effect of unnoticed redundancy – the original model has a redundant fact, and properly updating it requires adjusting both the redundant and non-redundant constraints.

Tasks 2 and 3 (observation). Participants examined two smaller models with redundant constraints. They were first asked to identify any redundancies manually, then to verify their assessment using the tool, and finally to use the Explain Redundancy feature on a selected redundant constraint.

Closing discussion. Participants were asked (i) whether they think redundancy in a model is good or bad, (ii) in which situations they would use the redundancy analysis, and (iii) their opinions on the explanation feature.

Task 1 directly demonstrates the maintenance risk from Sect. 2. Five of six participants initially updated only the first of the two semantically linked facts when extending the model with `Lecturer`, leaving the second unchanged. This caused models to produce no instances with lecturers. Only *E2*, who checked both facts before editing, identified both update sites.

When asked why two edits were necessary, participants agreed that the redundancy becomes essentially invisible during a local edit. As *E1* noted: “I didn’t really flag it as redundant on the first go through and didn’t pay attention to it when I added the lecturer. (...) it’s just more places you have to update.” This supports our hypothesis that redundancy reduces maintainability in a subtle way, even in small models.

In Tasks 2 and 3, participants were asked to identify redundancies without the tool. Results varied substantially by experience and by the type of redundancy.

For fact-to-fact redundancy, *E2* quickly identified redundancy by reasoning about logical implication, whereas *E1* and *N2* eventually identified it after extended analysis and instructor guidance. *N1*, *N3*, and *N4* either misunderstood or needed an explicit explanation before recognizing it. For structural redundancy, no participant identified the redundancy without prompting. *E2* correctly predicted that the second task contained a structural redundancy, but could not localize it; none of the novice participants noticed it at all. This aligns with Sect. 4.1 that structural constraints are hardest to reason about manually.

After using the tool, participants uniformly confirmed that the tool-detected redundancies were correct and that the tool provided value precisely in the cases where manual detection failed. *E2* remarked, “If I don’t understand why they are redundant, it means that something is bad.”

Participants found the Redundancy button easy to use across all six sessions. The tool’s syntax highlighting within the editor was noted positively. The Explain Redundancy feature had a usability issue – users needed to position the cursor within a redundant constraint before clicking, which was not clear from the UI. *E1* initially assumed the button would explain all redundancies at once – “I just thought I could hit the button and it would just explain any, like, all present redundancies.”

The explanation of redundant constraints was seen as useful, but with calibrated expectations. Participants compared it to an UNSAT core – useful for narrowing the search space but not a complete causal explanation. *E1*’s assessment was representative: “... it would give me a starting point, but it wouldn’t necessarily help me get all the way there to figuring out what I did wrong.” When redundancy came from structural elements, the tool reported: “no explanation based on other facts found; redundancy originates from structural elements.” This confused some novices (*N1*, *N3*, *N4*) who expected highlighted signatures or fields. They understood after it was explained, but the message itself did not provide actionable guidance.

All participants considered unintentional or undetected redundancy harmful. The most frequently cited concerns were maintainability and computational overhead. However, the study revealed a nuanced perspective among expert users, who noted that redundancy is not always entirely harmful and can serve as a validation mechanism. Participant *E1* explained that there is “a degree of safety in writing a specification to introduce something that’s redundant and check that it’s redundant,” using the tool to confirm that their mental model aligns with the specification’s logic.

The user study yields three main insights. First, redundancy causes maintenance challenges during model evolution. Second, manual redundancy detection is unreliable, especially for complex or structure-induced redundancies, emphasizing the need for automation. Third, participants found the tool’s redundancy detection useful and usable, but the explanation feature needs clearer guidance for structural redundancies and better discoverability of the cursor-based explanation workflow.

7 Conclusion

We have defined local (for a single command) and global (for all commands) redundancy of constraints in Alloy models. We have extended our definitions to maximal redundant subsets, i.e., subsets of constraints that can safely be removed without changing the model’s semantics.

We have presented algorithms to compute maximal redundant subsets of constraints, as well as explanations of redundant constraints, and implemented them in the Alloy Analyzer.

Our evaluation on a large, publicly available dataset shows that redundancy is prevalent in Alloy models and that it harms analysis times. We have identified trade-offs between a general explanation computation algorithm and one based on native UNSAT core calculation. A small, qualitative user study revealed that, on the one hand, redundancy poses maintenance challenges and is difficult to detect manually, but on the other hand, redundancy might be intentionally introduced. Our current tool support requires better explanation generation, in particular for explaining redundancy caused by (implicit) structural constraints.

Data Availability Statement

A complete replication package, comprising source code, datasets, and a Docker environment, is available at [37].

References

- [1] Mohannad Alhanahnah, Md Rashedul Hasan, Lisong Xu, and Hamid Bagheri. 2025. An empirical evaluation of pre-trained large language models for repairing declarative formal specifications. *Empir. Softw. Eng.* 30, 5 (2025), 149. doi:10.1007/S10664-025-10687-1
- [2] AlloyGitHub [n. d.]. Alloy Tools GitHub. <https://github.com/AlloyTools>. Accessed 3/2026.
- [3] AlloyWebsite [n. d.]. Alloy Tools website. <https://alloytools.org/>. Accessed 3/2026.
- [4] Hamid Bagheri, Eunsuk Kang, Sam Malek, and Daniel Jackson. 2018. A formal approach for detection of security flaws in the android permission system. *Formal Aspects Comput.* 30, 5 (2018), 525–544. doi:10.1007/S00165-017-0445-Z
- [5] Mira Balaban and Azzam Marae. 2019. Removing redundant multiplicity constraints in UML class models. *Softw. Syst. Model.* 18, 4 (2019), 2717–2751. doi:10.1007/S10270-018-0696-Z
- [6] Ilan Beer, Shoham Ben-David, Cindy Eisner, and Yoav Rodeh. 1997. Efficient Detection of Vacuity in ACTL Formulae. In *Computer Aided Verification, 9th International Conference, CAV '97, Haifa, Israel, June 22-25, 1997, Proceedings (Lecture Notes in Computer Science, Vol. 1254)*, Orna Grumberg (Ed.). Springer, 279–290. doi:10.1007/3-540-63166-6_28
- [7] Simón Gutiérrez Brida, Germán Regis, Guolong Zheng, Hamid Bagheri, ThanhVu Nguyen, Nazareno Aguirre, and Marcelo F. Frias. 2021. BeAFix: An Automated Repair Tool for Faulty Alloy Models. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 1213–1217. doi:10.1109/ASE51524.2021.9678524
- [8] Simón Gutiérrez Brida, Germán Regis, Guolong Zheng, Hamid Bagheri, ThanhVu Nguyen, Nazareno Aguirre, and Marcelo F. Frias. 2022. ICEBAR: Feedback-Driven Iterative Repair of Alloy Specifications. In *37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022*. ACM, 55:1–55:13. doi:10.1145/3551349.3556944
- [9] Julien Brunel, David Chemouil, Alcino Cunha, and Nuno Macedo. 2018. The electrum analyzer: model checking relational first-order temporal specifications. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018*, Marianne Huchard, Christian Kästner, and Gordon Fraser (Eds.). ACM, 884–887. doi:10.1145/3238147.3240475
- [10] Jorge Cerqueira, Alcino Cunha, and Nuno Macedo. 2022. Timely Specification Repair for Alloy 6. In *Software Engineering and Formal Methods - 20th International Conference, SEFM 2022, Berlin, Germany, September 26-30, 2022, Proceedings (Lecture Notes in Computer Science)*, Bernd-Holger Schlingloff and Ming Chai (Eds.). Springer, 288–303. doi:10.1007/978-3-031-17108-6_18
- [11] Hana Chockler and Ofer Strichman. 2007. Easier and More Informative Vacuity Checks. In *5th ACM & IEEE International Conference on Formal Methods and Models for Co-Design (MEMOCODE 2007), May 30 - June 1st, Nice, France*. IEEE Computer Society, 189–198. doi:10.1109/MEMCOD.2007.371225
- [12] Nima Dini, Cagdas Yelen, Zakaria Alrmaihi, Amresh Kulkarni, and Sarfraz Khurshid. 2018. Korat-API: a framework to enhance korat to better support testing and reliability techniques. In *SAC 2018*. ACM, 1934–1943. doi:10.1145/3167132.3167339
- [13] Tristan Dyer, Tim Nelson, Kathi Fisler, and Shriram Krishnamurthi. 2022. Applying cognitive principles to model-finding output: the positive value of negative information. *Proc. ACM Program. Lang.* 6, OOPSLA1 (2022), 1–29. doi:10.1145/3527323
- [14] Dana Fisman, Orna Kupferman, Sarai Sheinvald-Faragy, and Moshe Y. Vardi. 2008. A Framework for Inherent Vacuity. In *Hardware and Software: Verification and Testing, 4th International Haifa Verification Conference, HVC 2008, Haifa, Israel, October 27-30, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 5394)*, Hana Chockler and Alan J. Hu (Eds.). Springer, 7–22. doi:10.1007/978-3-642-01702-5_7
- [15] Javier Godoy, Juan Pablo Galeotti, Diego Garbervetsky, and Sebastián Uchitel. 2022. Predicate abstractions for smart contract validation. In *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems, MODELS 2022, Montreal, Quebec, Canada, October 23-28, 2022*, Eugene Syriani, Houari A. Sahraoui, Nelly Bencomo, and Manuel Wimmer (Eds.). ACM, 289–299. doi:10.1145/3550355.3552462
- [16] Daniel Jackson. 2002. Alloy: a lightweight object modelling notation. *ACM Trans. Softw. Eng. Methodol.* 11, 2 (2002), 256–290.
- [17] Daniel Jackson. 2006. *Software Abstractions: Logic, Language, and Analysis*. MIT Press.
- [18] Daniel Jackson. 2019. Alloy: a language and tool for exploring software designs. *Commun. ACM* 62, 9 (2019), 66–76. doi:10.1145/3338843
- [19] Ana Jovanovic and Allison Sullivan. 2023. Mutation Testing for Temporal Alloy Models. In *26th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2023, Västerås, Sweden, October 1-6, 2023*. IEEE, 228–238. doi:10.1109/MODELS58315.2023.00013
- [20] Ana Jovanovic and Allison Sullivan. 2024. Right or Wrong - Understanding How Users Write Software Models in Alloy. In *Software Engineering and Formal Methods - 22nd International Conference, SEFM 2024, Aveiro, Portugal, November 6-8, 2024, Proceedings (Lecture Notes in Computer Science)*, Alexandre Madeira and Alexander Knapp (Eds.). Springer, 309–327. doi:10.1007/978-3-031-77382-2_18
- [21] Ulrich Junker. 2004. QUICKXPLAIN: Preferred Explanations and Relaxations for Over-Constrained Problems. In *Proceedings of the Nineteenth National Conference on Artificial Intelligence, Sixteenth Conference on Innovative Applications of Artificial Intelligence, July 25-29, 2004, San Jose, California, USA*, Deborah L. McGuinness and George Ferguson (Eds.). AAAI Press / The MIT Press, 167–172. <http://www.aaai.org/Library/AAAI/2004/aaai04-027.php>
- [22] Tanvir Ahmed Khan, Allison Sullivan, and Kaiyuan Wang. 2021. AlloyFL: a fault localization framework for Alloy. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 1535–1539. doi:10.1145/3468264.3473116
- [23] Dor Ma’ayan, Shahar Maoz, and Jan Oliver Ringert. 2023. Anti-Patterns (Smells) in Temporal Specifications. In *45th IEEE/ACM International Conference on Software Engineering: New Ideas and Emerging Results, NIER@ICSE, Melbourne, Australia, May 14-20, 2023*. IEEE, 13–18. doi:10.1109/ICSE-NIER58687.2023.00009
- [24] Nuno Macedo, Julien Brunel, David Chemouil, and Alcino Cunha. 2022. Pardinus: A Temporal Relational Model Finder. *J. Autom. Reason.* 66, 4 (2022), 861–904. doi:10.1007/S10817-022-09642-2
- [25] Nuno Macedo, Julien Brunel, David Chemouil, Alcino Cunha, and Denis Kuperberg. 2016. Lightweight specification and analysis of dynamic systems with rich configurations. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13-18, 2016*, Thomas Zimmermann, Jane Cleland-Huang, and Zhendong Su (Eds.). ACM, 373–383. doi:10.1145/2950290.2950318
- [26] Nuno Macedo, Alcino Cunha, José Pereira, Renato Carvalho, Ricardo Silva, Ana C. R. Paiva, Miguel Sozinho Ramalho, and Daniel Castro Silva. 2021. Experiences on teaching alloy with an automated assessment platform. *Sci. Comput. Program.* 211 (2021), 102690. doi:10.1016/J.SCICO.2021.102690
- [27] Shahar Maoz and Rafi Shalom. 2020. Inherent vacuity for GR(1) specifications. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 99–110. doi:10.1145/3368089.3409669
- [28] Aleksandar Milicevic, Joseph P. Near, Eunsuk Kang, and Daniel Jackson. 2014. Alloy*: A higher-order relational constraint solver. (2014).
- [29] Timothy Nelson, Christopher Barratt, Daniel J. Dougherty, Kathi Fisler, and Shriram Krishnamurthi. 2010. The Margrave Tool for Firewall Analysis. In *LISA 2010*. USENIX Association. <https://www.usenix.org/conference/lisa10/margrave-tool-firewall-analysis>
- [30] Tim Nelson, Natasha Danas, Theophilos Giannakopoulos, and Shriram Krishnamurthi. 2019. Synthesizing Mutable Configurations: Setting up Systems for

- Success. In *34th IEEE/ACM International Conference on Automated Software Engineering Workshops, ASE Workshops 2019, San Diego, CA, USA, November 11–15, 2019*. IEEE, 81–85. doi:10.1109/ASEW.2019.00034
- [31] Jan Oliver Ringert and Allison Sullivan. 2023. Abstract alloy instances. In *International Symposium on Formal Methods*. Springer, 364–382.
- [32] Jan Oliver Ringert and Syed Waqee Wali. 2020. Semantic comparisons of Alloy models. In *MoDELS '20: ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems, Virtual Event, Canada, 18–23 October, 2020*. Eugene Syriani, Houari A. Sahraoui, Juan de Lara, and Silvia Abrahão (Eds.). ACM, 165–174. doi:10.1145/3365438.3410955
- [33] Hesam Samimi, Ei Darli Aung, and Todd D. Millstein. 2010. Falling Back on Executable Specifications. In *ECOOP 2010 (LNCS, Vol. 6183)*. Springer, 552–576. doi:10.1007/978-3-642-14107-2_26
- [34] Tushar Sharma and Diomidis Spinellis. 2018. A survey on software smells. *J. Syst. Softw.* 138 (2018), 158–173. doi:10.1016/j.jss.2017.12.034
- [35] Salla Siagne and Anis Tello. 2015. *JUnitMe2.0: JUnit Test Generation with Alloy and CodeModel*. Technical Report. University of Lille.
- [36] Soaibuzzaman, Salar Kalantari, and Jan Oliver Ringert. 2026. On Writing Alloy Models: Metrics and a new Dataset. In *Rigorous State-Based Methods (LNCS, Vol. 15728)*, Michael Leuschel and Fuyuki Ishikawa (Eds.). Springer Nature Switzerland, 70–87. doi:10.1007/978-3-031-94533-5_5
- [37] Soaibuzzaman and Jan Oliver Ringert. 2026. *On Redundancy in Alloy Models*. doi:10.5281/zenodo.21333886
- [38] Allison Sullivan, Kaiyuan Wang, and Sarfraz Khurshid. 2018. AUnit: A Test Automation Tool for Alloy. In *11th IEEE International Conference on Software Testing, Verification and Validation, ICST 2018, Västerås, Sweden, April 9–13, 2018*. IEEE Computer Society, 398–403. doi:10.1109/ICST.2018.00047
- [39] Allison Sullivan, Kaiyuan Wang, Raziheh Nokhbeh Zaeem, and Sarfraz Khurshid. 2017. Automated Test Generation and Mutation Testing for Alloy. In *2017 IEEE International Conference on Software Testing, Verification and Validation, ICST 2017, Tokyo, Japan, March 13–17, 2017*. IEEE Computer Society, 264–275. doi:10.1109/ICST.2017.31
- [40] Davide Taibi, Andrea Janes, and Valentina Lenarduzzi. 2017. How developers perceive smells in source code: A replicated study. *Inf. Softw. Technol.* 92 (2017), 223–235. doi:10.1016/j.infsof.2017.08.008
- [41] Oliver A. Tazl, Christopher Tafeit, Franz Wotawa, and Alexander Felfernig. 2022. DDMin versus QuickXplain - An Experimental Comparison of two Algorithms for Minimizing Collections. In *The 34th International Conference on Software Engineering and Knowledge Engineering, SEKE 2022, KSIR Virtual Conference Center, USA, July 1 - July 10, 2022*, Rong Peng, Carlos Eduardo Pantoja, and Pankaj Kamthan (Eds.). KSI Research Inc., 481–486. doi:10.18293/SEKE2022-172
- [42] Nicola Thoben and Heike Wehrheim. 2025. Detecting Redundant Preconditions. In *13th IEEE/ACM International Conference on Formal Methods in Software Engineering, FormaliSE@ICSE 2025, Ottawa, ON, Canada, April 27–28, 2025*. IEEE, 86–95. doi:10.1109/FORMALISE66629.2025.00015
- [43] Emina Torlak, Felix Sheng-Ho Chang, and Daniel Jackson. 2008. Finding Minimal Unsatisfiable Cores of Declarative Specifications. In *FM 2008: Formal Methods, 15th International Symposium on Formal Methods, Turku, Finland, May 26–30, 2008, Proceedings (Lecture Notes in Computer Science, Vol. 5014)*, Jorge Cuéllar, T. S. E. Maibaum, and Kaisa Sere (Eds.). Springer, 326–341. doi:10.1007/978-3-540-68237-0_23
- [44] Emina Torlak and Daniel Jackson. 2007. Kodkod: A Relational Model Finder. In *Tools and Algorithms for the Construction and Analysis of Systems, 13th International Conference, TACAS 2007, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2007 Braga, Portugal, March 24 - April 1, 2007, Proceedings (Lecture Notes in Computer Science)*, Orna Grumberg and Michael Huth (Eds.). Springer, 632–647. doi:10.1007/978-3-540-71209-1_49
- [45] Caroline Trippel, Daniel Lustig, and Margaret Martonosi. 2019. Security Verification via Automatic Hardware-Aware Exploit Synthesis: The CheckMate Approach. *IEEE Micro* 39, 3 (2019), 84–93. doi:10.1109/MM.2019.2910010
- [46] Dániel Varró, Gábor Bergmann, Ábel Hegedüs, Ákos Horváth, István Ráth, and Zoltán Ujhelyi. 2016. Road to a reactive and incremental model transformation platform: three generations of the VIATRA framework. *Softw. Syst. Model.* 15, 3 (2016), 609–629. doi:10.1007/S10270-016-0530-4
- [47] Kaiyuan Wang, Allison Sullivan, and Sarfraz Khurshid. 2019. ARepair: a repair framework for alloy. In *Proceedings of the 41st International Conference on Software Engineering: Companion Proceedings, ICSE 2019, Montreal, QC, Canada, May 25–31, 2019*, Joanne M. Atlee, Tevfik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 103–106. doi:10.1109/ICSE-COMPANION.2019.00049
- [48] Kaiyuan Wang, Allison Sullivan, Darko Marinov, and Sarfraz Khurshid. 2020. Fault Localization for Declarative Models in Alloy. In *31st IEEE International Symposium on Software Reliability Engineering, ISSRE 2020, Coimbra, Portugal, October 12–15, 2020*, Marco Vieira, Henrique Madeira, Nuno Antunes, and Zheng Zheng (Eds.). IEEE, 391–402. doi:10.1109/ISSRE5003.2020.00044
- [49] Guanxuan Wu and Allison Sullivan. 2024. AlloyASG: Alloy Predicate Code Representation as a Compact Structurally Balanced Graph. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, MODELS 2024, Linz, Austria, September 22–27, 2024*, Alexander Egyed, Manuel Wimmer, Marsha Chechik, and Benoit Combemale (Eds.). ACM, 57–68. doi:10.1145/3640310.3674088
- [50] Raziheh Nokhbeh Zaeem and Sarfraz Khurshid. 2010. Contract-Based Data Structure Repair Using Alloy. In *ECOOP 2010 (LNCS, Vol. 6183)*. Springer, 577–598. doi:10.1007/978-3-642-14107-2_27
- [51] Pamela Zave. 2017. Reasoning About Identifier Spaces: How to Make Chord Correct. *IEEE Trans. Software Eng.* 43, 12 (2017), 1144–1156. doi:10.1109/TSE.2017.2655056
- [52] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Trans. Software Eng.* 28, 2 (2002), 183–200. doi:10.1109/32.988498
- [53] Guolong Zheng, ThanhVu Nguyen, Simón Gutiérrez Brida, Germán Regis, Marcelo F. Frias, Nazareno Aguirre, and Hamid Bagheri. 2021. FLACK: Counterexample-Guided Fault Localization for Alloy Models. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22–30 May 2021*. IEEE, 637–648. doi:10.1109/ICSE43902.2021.00065